

Ali Alemami

Amman, Jordan | ali.alemami01@gmail.com | +962 780 020 203 | github.com/ali-alemami | ali-alemami.github.io | linkedin.com/in/ali-alemami

Computer Science student specializing in backend & low-level systems engineering — rigorous practical experience in **C/C++** (POSIX kernel APIs, AST parsing, multithreading, concurrency, memory models) and **C#/ .NET** enterprise software architecture (3-tier design, ADO.NET, SQL Server). Seeking a backend, systems, or software engineering role.

EDUCATION

The Hashemite University — BSc in Computer Science 2022 – present
42 Amman — Common Core, Level 4.7/21, 21/24 core projects complete (peer-reviewed, project-based) 2025 – present

TECHNICAL SKILLS

Languages: C (POSIX/C99), C++ (98/11/17), C# (.NET), SQL (T-SQL), Bash
Systems & Architecture: Unix process control, AST parsing, multithreading (pthreads), mutex synchronization, sockets, 3-tier architecture, IPC
CS Fundamentals: OOP & Design Patterns, Data Structures & Algorithms, Memory Management (Valgrind/ASan), Computer Networks
Database & Tools: SQL Server, ADO.NET, Docker, Docker Compose, NGINX, Git, Linux, GDB, Valgrind, Makefiles

42 CORE CURRICULUM — 21/24 PROJECTS COMPLETE

- **Multithreading & Concurrency (C — philo):** Simulated Dijkstra's Dining Philosophers problem using POSIX threads (pthreads) and mutex locks; implemented microsecond-precision state monitoring to guarantee zero data races, starvation prevention, and deadlock-free synchronization.
- **DevOps & Container Infrastructure (inception):** Orchestrated a secure multi-service infrastructure using Docker Compose; configured isolated containers for NGINX with TLS/SSL encryption, WordPress (PHP-FPM), and MariaDB with dedicated storage volumes, non-root daemons, and internal networking.
- **Unix Shell & AST Engine (C — minishell):** Engineered a POSIX-compliant shell with an Abstract Syntax Tree (AST) parser to evaluate operator precedence ([, &&, |]), nested subshell forks, heredocs, variable expansions, and signal-safe process control; audited for zero fd leaks via Valgrind.
- **3D Graphics & Raytracing (C — mini_rt):** Built a 3D raytracing engine from scratch in C; implemented vector linear algebra, ray-primitive geometric intersections (spheres, planes, cylinders), and Phong illumination physics (ambient, diffuse, specular lighting, and hard shadows) rendered via MiniLibX.
- **C++ Modules & Ford-Johnson Algorithm (C++ — cpps):** Completed 42 C++ Modules 00–09 (OOP, Orthodox Canonical Form, polymorphism, templates, STL); implemented the **Ford-Johnson Merge-Insert Sort algorithm (CPP09)** optimizing comparison counts using multiple STL sequence containers (std::vector, std::deque) and Jacobsthal sequence partitioning.
- **Foundations & Algorithms:** [push_swap](#) (two-stack sorting optimization), [pipex](#) (pipeline fork/dup2), [fract-ol](#) (Mandelbrot/Julia sets), [minitalk](#) (signal IPC), [libft](#), [ft_printf](#), [get_next_line](#). *In progress:* [webserv](#) (C++98 HTTP/1.1 epoll server), [transcendence](#).

PROGRAMMINGADVICES ROADMAP — 17/24 COURSES COMPLETE

Dr. Mohammed Abu-Hadhoud curriculum — software architecture, advanced problem solving, database engineering, and .NET

- **Bank & ATM Management Engine (C++, 2,000+ lines — Bank-and-ATM-System-CPP):** Engineered a modular, dual-engine banking & ATM system in C++; implemented bitmask-based user role permissions (bitwise flags), custom delimiter-based file serialization (###) for data persistence, and transaction audit logs.
- **Contacts 3-Tier Enterprise Application (C#, ADO.NET, SQL Server — Contacts-App-3Tier):** Architected an enterprise application with strict separation of concerns across Presentation, Business Logic, and Data Access layers; used parameterized T-SQL commands to enforce data integrity and prevent SQL injection.
- **Relational Database Schemas & SQL Engineering (SQL Server — Programming-Advices-Core):** Designed 5 relational database schemas (Medical Clinic Booking, E-Commerce Store, HR/Payroll, Vehicle Registry) with primary/foreign key constraints, normalization, and complex multi-table analytical JOIN queries.
- **Algorithmic Master Repository (C++ • 225+ Problem Solutions / 48k+ lines):** Engineered solutions across 5 levels of computational problem solving (matrix operations, string manipulation, combinatorial math, and calendar/date arithmetic engines).
- **Windows Forms Event-Driven Suite (C#, .NET • 11.5k+ lines — CSharp-WinForms-Collection):** Developed 10+ desktop applications in C# implementing real-time UI state evaluation, input validation, and asynchronous event handling.